

09 — Key-Compromise Response Runbook

What to do if `WALLET_DERIVATION_SECRET` or `ENCRYPTION_KEY` is suspected exposed. Because every user wallet derives from the master secret, exposure is the **highest-severity** incident. Print this; don't improvise it under stress.

Severity & owners

- **Severity: CRITICAL (P0).** Potential total loss of user funds.
- **Incident commander:** [name / role]
- **Backup:** [name]
- **Notify immediately:** CTO, CEO, Compliance/MLRO, on/off-ramp partner contact.

Triggers (any of)

- Master secret seen in a log, error report, screenshot, chat, commit, or backup.
- A host with access to the secret is compromised or behaves anomalously.
- Unexplained outbound transfers / balance drops across multiple users.
- A departing insider had access and access wasn't revoked.

Step 1 — CONTAIN (minutes)

1. **Freeze movement.** Put the platform into a state where **no outbound transfers are processed** (disable send/withdraw/remit/agent-payout — e.g. maintenance flag / pause the signing path).
2. **Rotate everything reachable now:** partner API keys, `JWT_SECRET`, DB credentials, admin/staff sessions (force re-login). This limits lateral use.
3. **Revoke access** to the secret: rotate cloud credentials, pull the secret from any host that doesn't strictly need it, disable suspected accounts.
4. **Preserve evidence:** snapshot logs, access logs, and the `AdminAuditLog` (incl. KYC-access trail) before anything is wiped.

Step 2 — ASSESS (first hour)

- Confirm whether the secret was actually exposed vs. suspected.
- Check on-chain: pull recent transfers for platform + user wallets; identify any unauthorised movements (Horizon).
- Quantify exposure window (when could the secret have leaked → now).

Step 3 — SWEEP MIGRATION (if exposure confirmed)

Because addresses derive from the secret, you cannot "rotate in place" — you **migrate funds to wallets under a NEW secret**: 1. Generate a **new** `WALLET_DERIVATION_SECRET` inside the secure boundary (KMS/HSM if available — see Doc 08). 2. For each user, derive the **old** key (old secret + phone) **inside the boundary** and the **new** address (new secret + phone). 3. **Sweep** each user's USDC (and dust XLM) from old → new address on-chain, in controlled batches, sponsored by the gas wallet. Log every move. 4. Update each user's stored `stellarPubKey` (+ re-encrypt secret under new scheme) to the new address. 5.

Reconcile: sum of swept balances == pre-sweep liabilities. Investigate any gap. 6. Keep the old secret **only** until the sweep is verified complete, then destroy.

Pre-write and table-top test this sweep script **before** you ever need it. Doing it for the first time during a live incident is how funds get lost.

Step 4 — RESTORE

- Re-enable transfers only after: new secret in place, sweep verified, reconciliation balanced, and the exposure vector closed.
- Force a fresh app session for all users; confirm balances display correctly.

Step 5 — NOTIFY & REPORT

- **Users:** clear, honest notice if funds/PII were at risk (per law).
- **Partner (Yellow Card):** per your agreement's incident clause.
- **Regulator / FIU:** breach notification per applicable Tanzanian law and AML/CFT obligations (MLRO leads). Timelines: [confirm with counsel].
- **Data protection:** if PII (KYC docs) exposed, follow breach-notification law.

Step 6 — POST-INCIDENT

- Root-cause analysis; close the vector (e.g. move secret to KMS — Doc 08).
- Update this runbook with lessons learned.
- Independent review of the response.

Standing prevention checklist (do these now — Phase 0)

- [] Master secrets excluded from logs, Sentry (scrubbing in place), backups, bundles.
- [] Alert on access to / change of the master secrets.
- [] Least-privilege access; list exactly who/what can read them.
- [] Sweep-migration script written + table-top tested.
- [] This runbook printed and owners assigned.
- [] Roadmap: move signing into a KMS/HSM-backed signer (Doc 08).